

머신러닝 프로그래밍

Machine Learning Programming

Lecture 9 : Decision Tree



충북대학교
CHUNGBUK NATIONAL UNIVERSITY



충북대학교
산업인공지능연구센터

의사결정트리의 이해

의사결정트리의 개념

Decision tree : 어떤 규칙을 하나의 트리(tree) 형태로 표현한 후 이를 바탕으로 분류나 회귀 문제를 해결

- 규칙은 'if-else' 문으로 표현이 가능
- 트리는 일종의 경로를 표현하는 것
- 트리 구조의 마지막 노드에는 분류 문제에서 클래스, 회귀 문제에서는 예측치가 들어감

의사결정트리의 이해

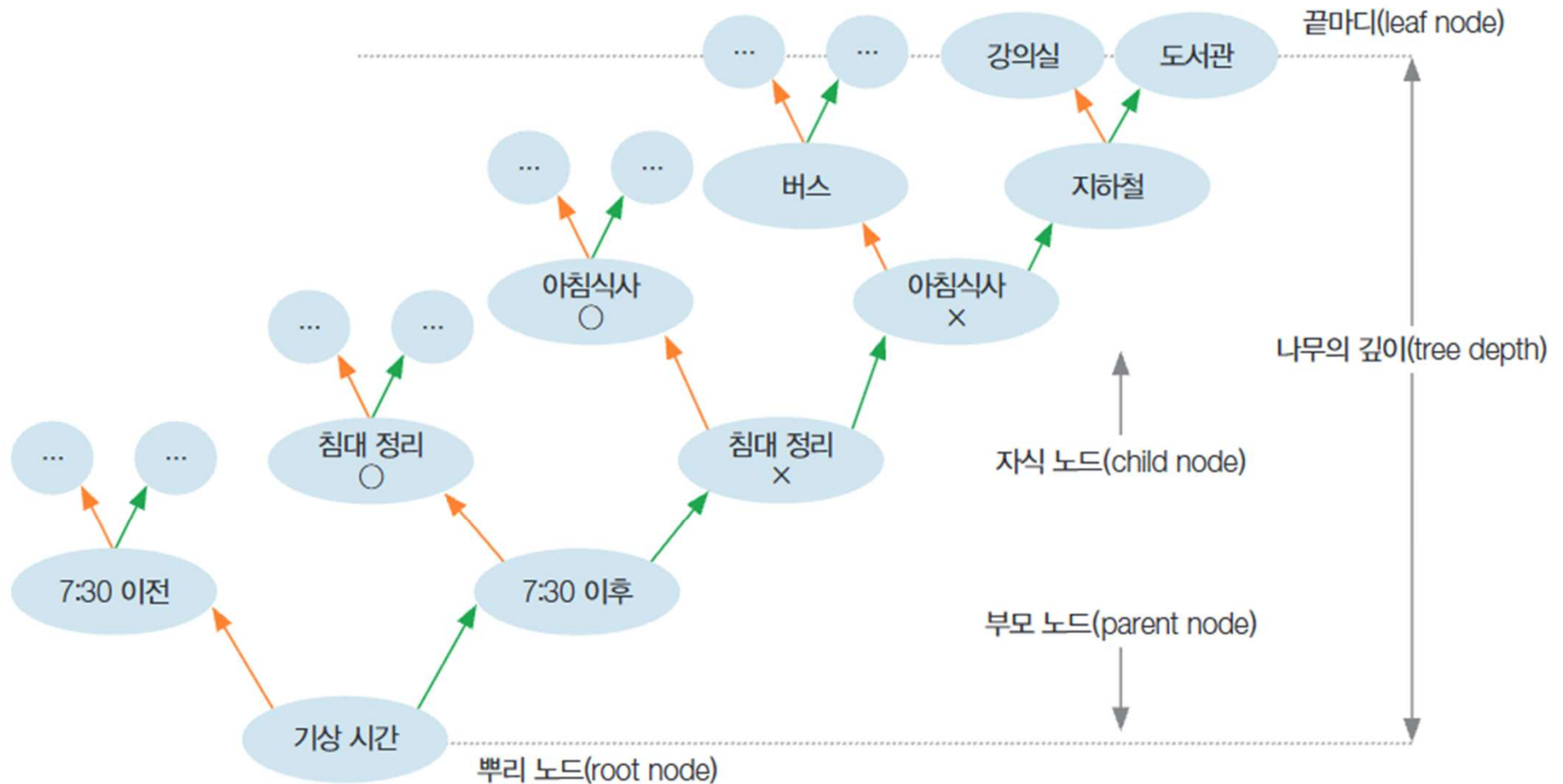


그림1. 트리를 구성하는 각 부분의 명칭

의사결정트리의 이해



그림2. 아키네이터 게임

의사결정트리의 이해

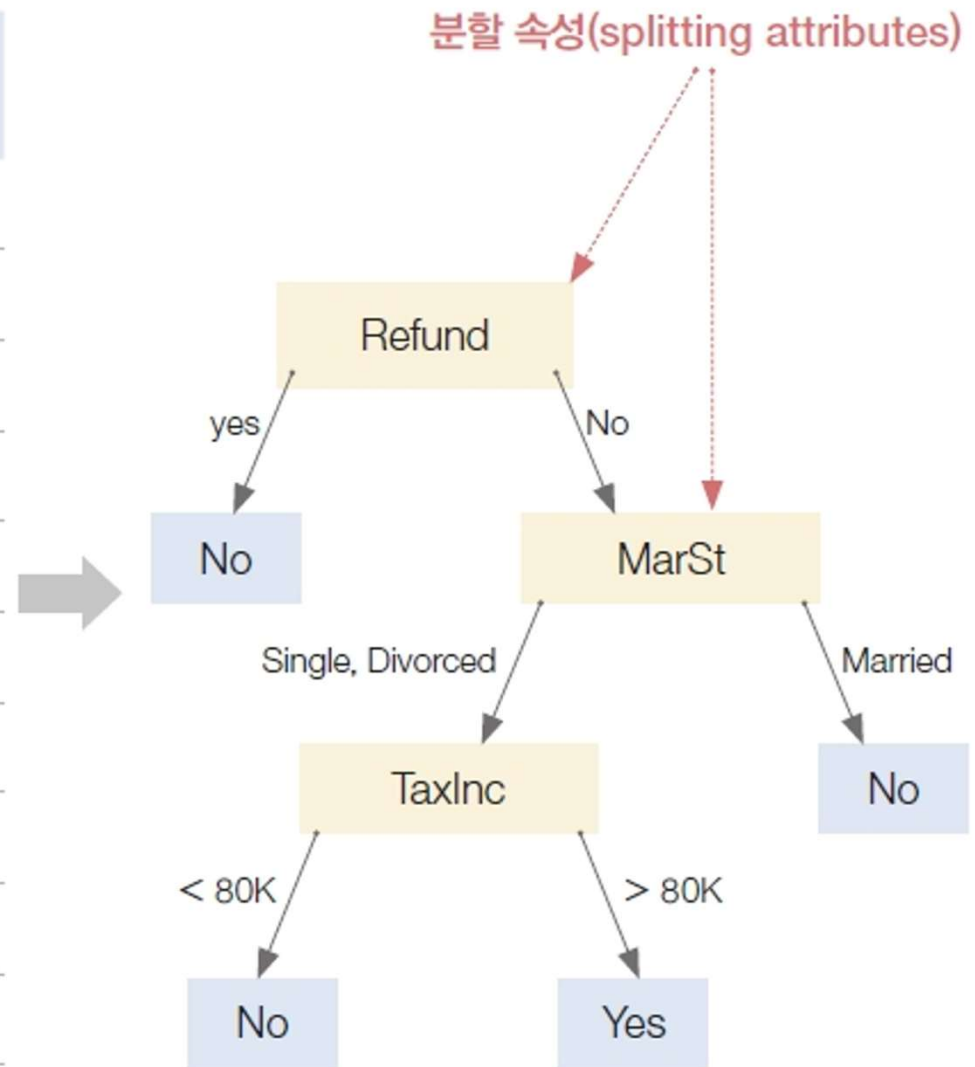
의사결정트리는 딥러닝 기반을 제외한 전통적인 통계 기반의 머신러닝 모델 중 효과와 실용성이 가장 좋음

- 테이블형 데이터에 있어 설명력과 성능의 측면에서 딥러닝 모델들과 대등하게 경쟁
- 앙상블(ensemble) 모델이나 부스팅(boosting) 같은 새로운 기법들이 모델들의 성능을 대폭 향상시키고 있음

의사결정트리의 이해

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

(a) 훈련 데이터(training data)



(b) 의사결정트리(decision tree)

그림3. 의사결정트리 만들기

의사결정트리의 이해

엔트로피의 이해

- 엔트로피 : 어떤 목적 달성을 위한 경우의 수를 정량적으로 표현하는 수치
 - 현재의 정보 제공 상태를 측정
 - 어떤 분할 속성을 선택하였을 때 정보를 제공하는 기준 값을 정하고, 그 값을 최소화 또는 최대화하는 방향으로 알고리즘 실행
- 낮은 엔트로피 = 경우의 수가 적음 = 낮은 불확실성
- 높은 엔트로피 = 경우의 수가 높음 = 높은 불확실성

의사결정트리의 이해

- 엔트로피를 측정하는 방법 : 샤논(Shannon, Claude Elwood)이라는 공식을 사용

$$h(D) = -\sum_{i=1}^m p_i \log_2(p_i) \quad \text{where} \begin{cases} D & \text{Data set} \\ p_i & \text{Probability of label } i \end{cases}$$

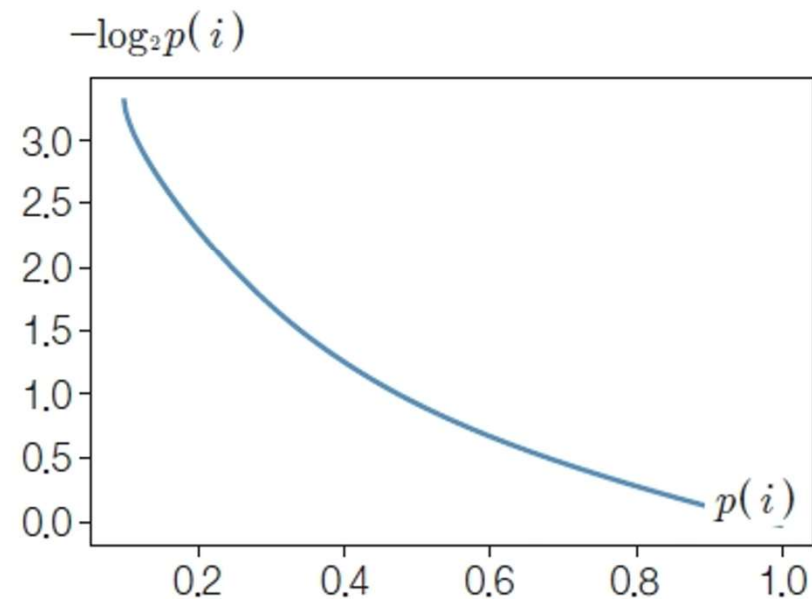
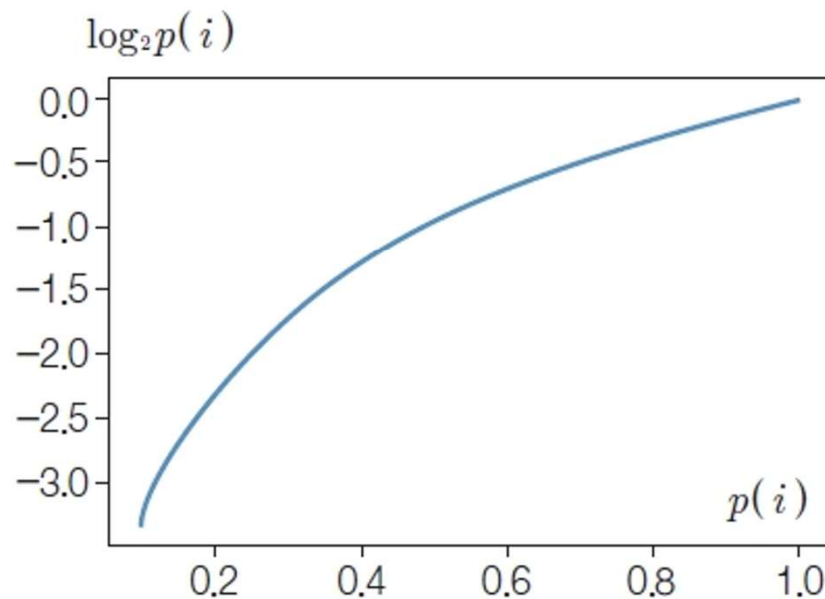


그림4. 샤논의 공식을 그래프로 표현

의사결정트리의 이해

엔트로피의 예시

표 1. 예시 데이터

No	age	income	student	credit_rating	class_buys_computer
0	youth	high	no	fair	no
1	youth	high	no	excellent	no
2	middle_aged	high	no	fair	yes
3	senior	medium	no	fair	yes
4	senior	low	yes	fair	yes
5	senior	low	yes	excellent	no
6	middle_aged	low	yes	excellent	yes
7	youth	medium	no	fair	no
8	youth	low	yes	fair	yes
9	senior	medium	yes	fair	yes
10	youth	medium	yes	excellent	yes
11	middle_aged	medium	no	excellent	yes
12	middle_aged	high	yes	fair	yes
13	senior	medium	no	excellent	no

의사결정트리의 이해

- 사람의 나이, 소득, 학생 여부, 신용 등급 등을 고려하여 컴퓨터를 구매할지 구매하지 않을지 의사결정
- 컴퓨터를 구매할 확률 $\frac{9}{14}$, 구매하지 않을 확률 $\frac{5}{14}$

$$h(D) = -\sum_{i=1}^m p_i \log_2(p_i)$$

$$h(d) = -\frac{9}{14} \log_2 \frac{9}{14} + \left(-\frac{5}{14} \log_2 \frac{5}{14} \right) = 0.940$$

의사결정트리 알고리즘

정보 이득

- 정보 이득(information gain) : 엔트로피를 사용하여 속성별 분류 시 데이터가 얼마나 순수한지(impurity)를 측정하는 지표
 - 각 속성을 기준으로 데이터를 분류했을 때 엔트로피를 측정

$$\begin{matrix} \text{전체 엔트로피} & - & \text{속성별 엔트로피} & = & \text{속성별 정보 이득} \\ \textcircled{1} & & \textcircled{2} & & \textcircled{3} \end{matrix}$$

① 전체 엔트로피

$$Info(D) = - \sum_{i=1}^n p_i \log_2(p_i)$$

의사결정트리 알고리즘

② 속성별 엔트로피

$$Info_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

- 속성 A로 데이터를 분류했을 때 속성 A가 가진 모든 클래스의 각 엔트로피를 계산한 후, 데이터의 개수만큼 가중치를 줌

③ 속성별 정보 이득 :

$$Gain(A) = Info(D) - Info_A(D)$$

- 정보 이득이 크면 클수록 A를 기준으로 데이터를 분류했을 때 얻을 수 있는 정보량이 많다는 뜻
- A를 기준으로 데이터를 나눌 때 엔트로피가 작다면 해당 속성을 기준으로 데이터를 나누기 좋다고 볼 수 있음

의사결정트리 알고리즘

ID3 알고리즘을 활용한 의사결정트리의 성장

- 성장(grow) : 일반적으로 의사결정트리를 생성하는 방법을 성장이라고 부름. 트리(나무)를 성장시키는 개념
- ID3(Iterative Dichotomiser 3) : 반복적으로(iteratively) 데이터를 나누는(divides) 알고리즘
 - 톱다운(top-down) 방식으로 데이터를 나누면서 탐욕적(greedy)으로 현재 상태에서 최적화를 추진하는 방법을 선택

의사결정트리 알고리즘

표 2. 예시 데이터

No	age	income	student	credit_rating	class_buys_computer
0	youth	high	no	fair	no
1	youth	high	no	excellent	no
2	middle_aged	high	no	fair	yes
3	senior	medium	no	fair	yes
4	senior	low	yes	fair	yes
5	senior	low	yes	excellent	no
6	middle_aged	low	yes	excellent	yes
7	youth	medium	no	fair	no
8	youth	low	yes	fair	yes
9	senior	medium	yes	fair	yes
10	youth	medium	yes	excellent	yes
11	middle_aged	medium	no	excellent	yes
12	middle_aged	high	yes	fair	yes
13	senior	medium	no	excellent	no

의사결정트리 알고리즘

- 모든 데이터가 동일한 클래스가 아님
- 최적 분류 기준이 되는 속성을 선정하기 위해, 정보 이득을 기반으로 속성별 데이터 분류의 기준을 정함

age	$Gain(age) = Info(D) - Info_{age}(D)$
credit	$Gain(credit) = Info(D) - Info_{credit}(D)$
income	$Gain(income) = Info(D) - Info_{income}(D)$
student	$Gain(student) = Info(D) - Info_{student}(D)$

$$Info(D) = -\sum_{i=1}^n p_i \log_2(p_i) = \left(-\frac{9}{14}\right) \log_2 \frac{9}{14} + \left(-\frac{5}{14}\right) \log_2 \frac{5}{14} = 0.940$$

의사결정트리 알고리즘

age 기준의 정보 이득

$$Info_{age}(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

- j는 age 속성의 클래스인 youth, middle_age, senior

$$\begin{aligned} Info_{age}(D) &= \frac{5}{14} \times \left(-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \right) \\ &\quad + \frac{4}{14} \times \left(-\frac{4}{4} \log_2 \frac{4}{4} \right) \\ &\quad + \frac{4}{14} \times \left(-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \right) \end{aligned}$$

- $Gain(age)$ 는 0.24674

의사결정트리 알고리즘

다른 속성들의 정보 이득과 가지 속성

age	$Gain(age) = Info(D) - Info_{age}(D) = 0.24674$
credit	$Gain(credit) = Info(D) - Info_{credit}(D) = 0.02922$
income	$Gain(income) = Info(D) - Info_{income}(D) = 0.15183$
student	$Gain(student) = Info(D) - Info_{student}(D) = 0.04812$

- ID3 알고리즘의 순서에 따라 가장 많은 정보를 주는 age 속성을 첫 번째 가지(branch) 속성이라고 함

의사결정트리 알고리즘

- age 속성 기준으로 데이터를 나누어 새로운 트리 생성

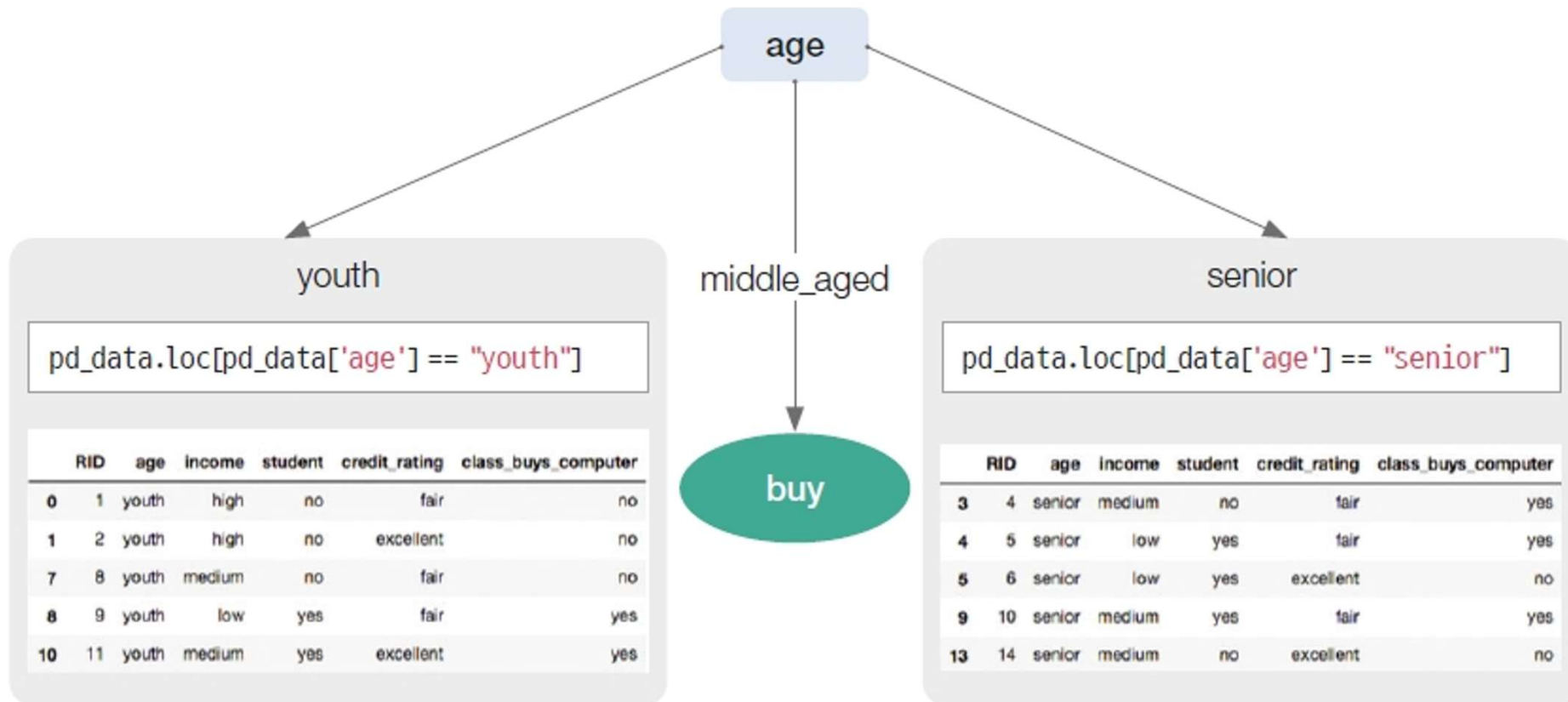


그림 5. age 속성을 기준으로 한 트리(tree) 생성

의사결정트리 알고리즘

- youth와 senior는 3:2 비율로 컴퓨터 구매 여부가 나누어짐
- middle_aged의 경우 모두 컴퓨터를 구입한다는 데이터이므로 더 이상 데이터를 분류할 필요가 없음

■ age가 youth로 분류된 5개의 데이터에 대한 정보 이득

기본 불확실성 (엔트로피):

$$\bullet \text{ } Info(D_{youth}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \approx 0.971$$

1. student 기준 분할 (Gain: 0.971)

$$\bullet \text{ } Info_{student}(D_{youth}) = \frac{2}{5}(0) + \frac{3}{5}(0) = 0$$

$$\bullet \text{ } Gain(student) = Info(D_{youth}) - Info_{student}(D_{youth}) = 0.971$$

2. income 기준 분할 (Gain: 0.571)

$$\bullet \text{ } Info_{income}(D_{youth}) = \frac{2}{5}(0) + \frac{2}{5}(1) + \frac{1}{5}(0) = 0.400$$

$$\bullet \text{ } Gain(income) = Info(D_{youth}) - Info_{income}(D_{youth}) = 0.571$$

3. credit_rating 기준 분할 (Gain: 0.020)

$$\bullet \text{ } Info_{credit_rating}(D_{youth}) = \frac{3}{5}(0.918) + \frac{2}{5}(1.0) = 0.951$$

$$\bullet \text{ } Gain(credit_rating) = Info(D_{youth}) - Info_{credit_rating}(D_{youth}) = 0.020$$

의사결정트리 알고리즘

- student를 기준으로 데이터를 분리했을 때 가장 많은 정보를 획득함

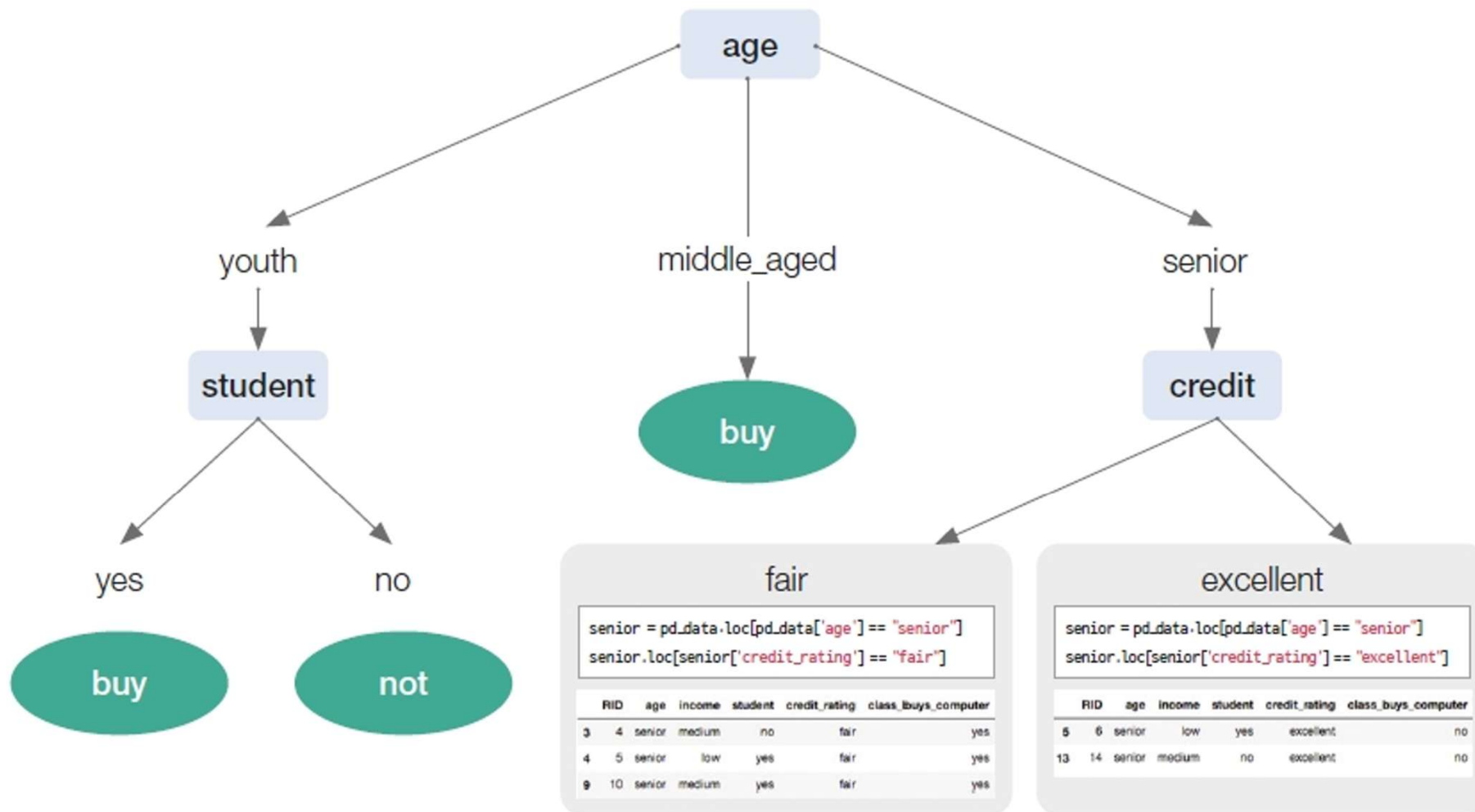


그림 6. 데이터 재분류

의사결정트리 알고리즘

- 의사결정트리 완성

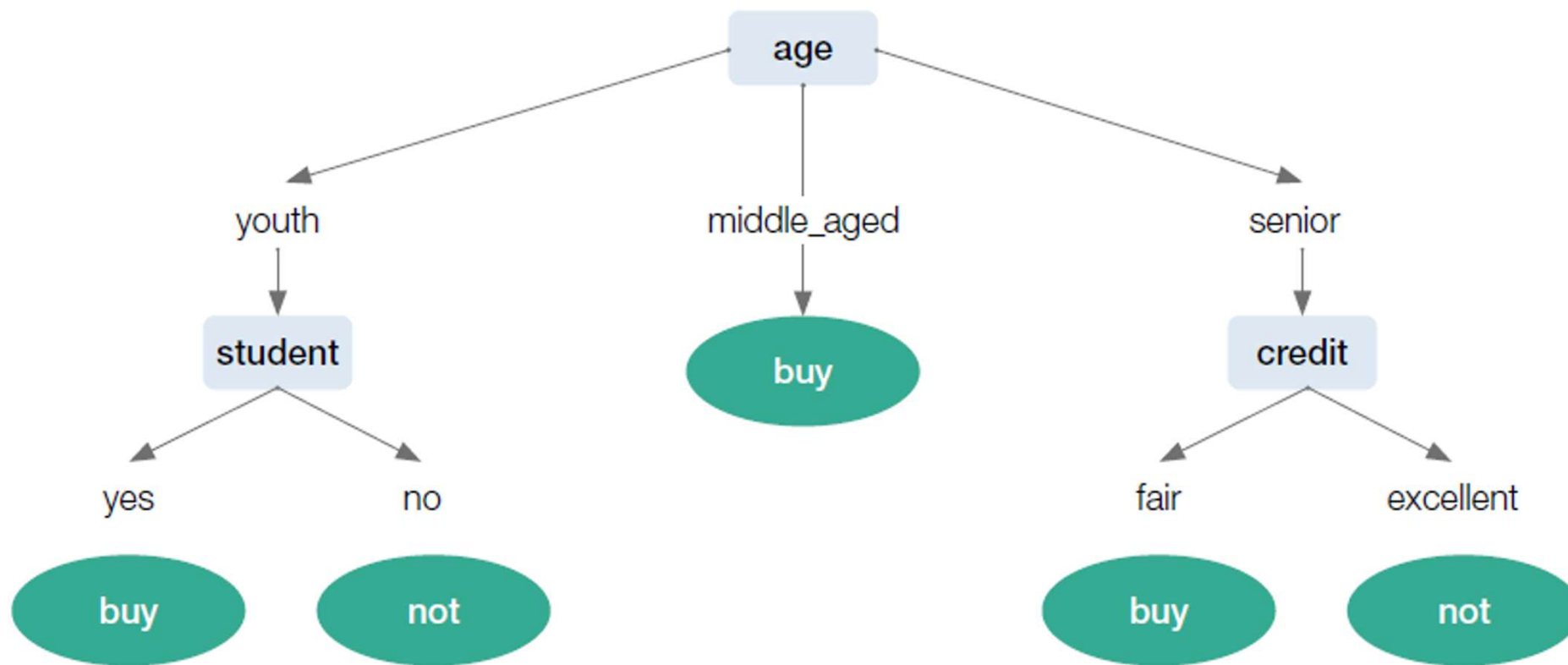


그림 7. 의사결정트리 완성

의사결정트리 알고리즘

의사결정트리 알고리즘의 특징

- ① 재귀적 작동 : 가지가 되는 속성을 선택한 후 해당 가지로 데이터를 나누면, 이전에 적용되었던 알고리즘이 남은 데이터에 적용됨
 - 남은 데이터에서만 최적의 모델을 찾는 방법으로 작동
- ② 속성 기준으로 가지치기 수행 : 가장 불확실성이 적은 속성을 기준으로 가지치기를 수행
- ③ 중요한 속성 정보 제공 : 처음 분리 대상이 되는 속성이 가장 중요한 속성
 - 이 특징을 '해석 가능한 머신러닝'이라고 부름

의사결정트리 알고리즘

의사결정트리의 장점

- 불필요한 속성 값에 대한 스케일링 : 전처리 단계 없이 바로 사용할 수 있다
- 강건(robust)한 이상치(outlier) : 관측치의 절대값이 아닌 순서가 중요하기 때문에 필요 이상으로 엄청 큰 값이나 작은 값에 대해서도 분류 성능이 크게 떨어지지 않는다.
- 자동적인 변수 선택 : 알고리즘에 의해 중요한 변수들이 우선적으로 선택되어 조금 더 손쉽게 중요한 속성을 확인할 수 있다. 의사결정 트리 계열의 알고리즘이 가지고 있는 가장 큰 장점 중 하나이다.

의사결정트리의 확장

정보 이득의 문제점

- 수식의 특성상 속성의 값이 다양할수록 선택의 확률이 높아지는 문제가 발생

$$Info_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

- 데이터가 매우 많고 속성이 다양할 때 위 수식의 $\frac{|D_j|}{|D|}$ 값이 작아짐. 해당 속성의 엔트로피가 낮아져 단순히 속성 안에 있는 값의 종류를 늘리는 것만으로 정보 이득이 높아짐

의사결정트리의 확장

C4.5 알고리즘

- C4.5 : 정보 이득을 측정하는 방식을 좀 더 평준화시켜 단순한 정보 값을 대신 사용
 - 기존 정보 이득의 분모에 평준화 함수 SplitInfo 추가

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)} = \frac{Info(D) - Info_a(D)}{SplitInfo_A(D)}$$

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \left(\log_2 \frac{|D_j|}{|D|} \right)$$

의사결정트리의 확장

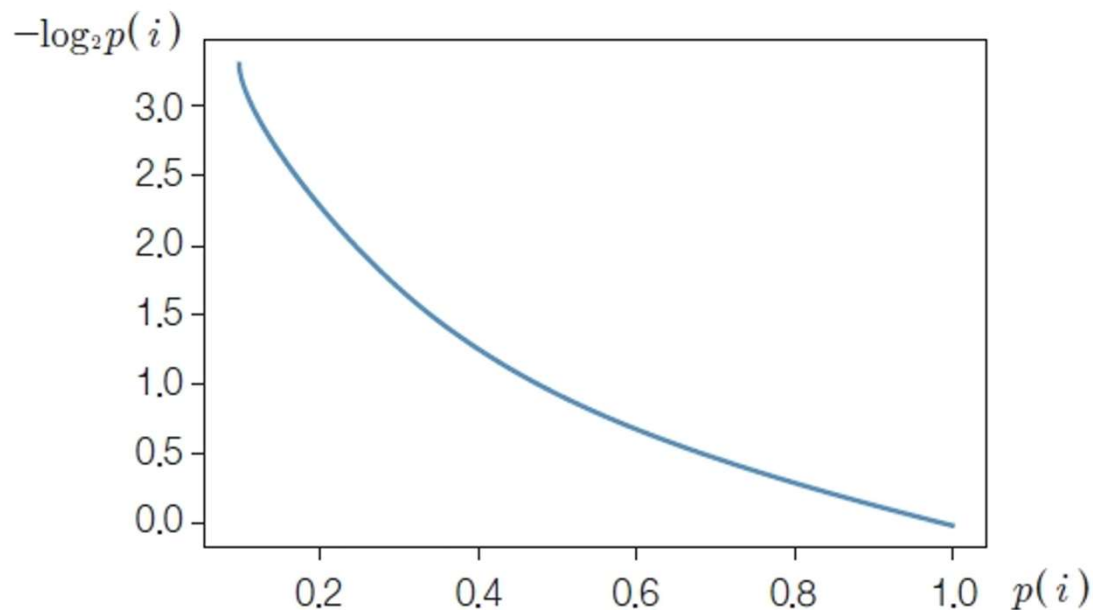


그림 8. C4.5 알고리즘

- 클래스가 많을수록 $\frac{|D_j|}{|D|}$ 값이 작아지고 $-\log_2 \frac{|D_j|}{|D|}$ 값은 커져 정규화됨
- SplitInfo 함수 값이 분모에 들어가면서 고유값이 지나치게 많은 다중 분할 속성(Multi-way split attribute)에 대한 페널티 부여

의사결정트리의 확장

지니 지수

- 경제학에서 소득의 불평등도를 측정할 때 사용하는 지표인데, 의사결정트리에서 각 속성의 불순도를 측정하는 방법으로 사용

$$Gini(D) = 1 - \sum_{i=1}^m p_i^2 = 1 - \sum_{i=1}^m \frac{|C_{i,D}|}{|D|} \text{ where } C_i \text{ is a class}$$

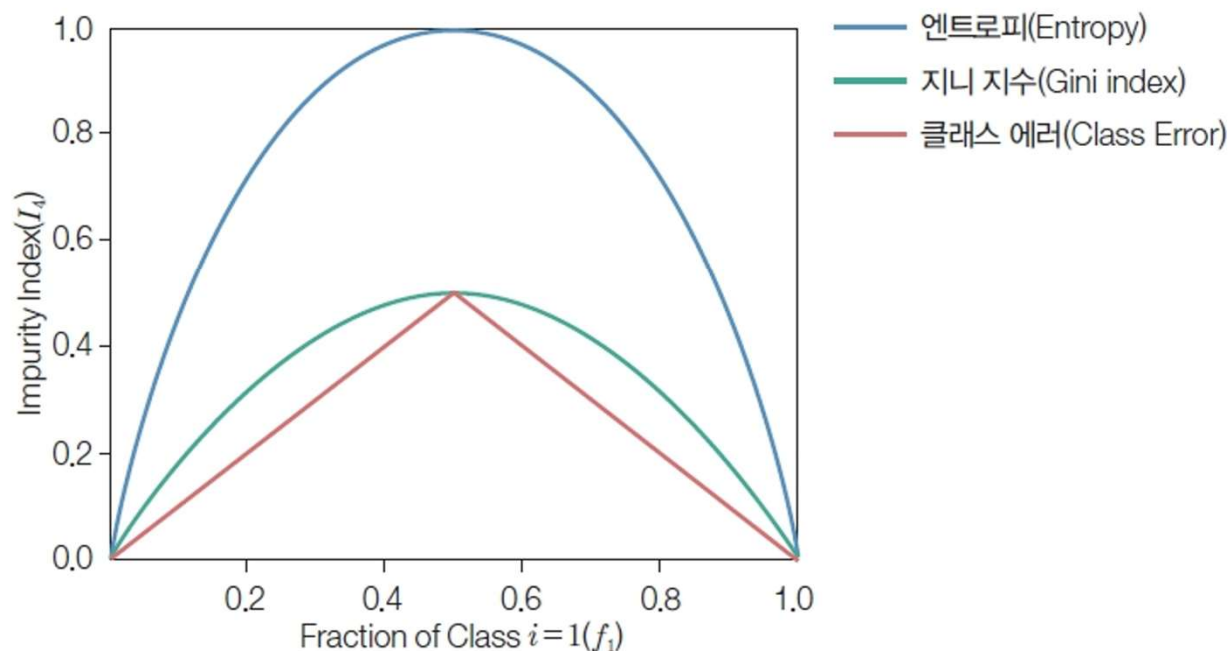


그림 9. 지니 지수로 작성된 그래프

의사결정트리의 확장

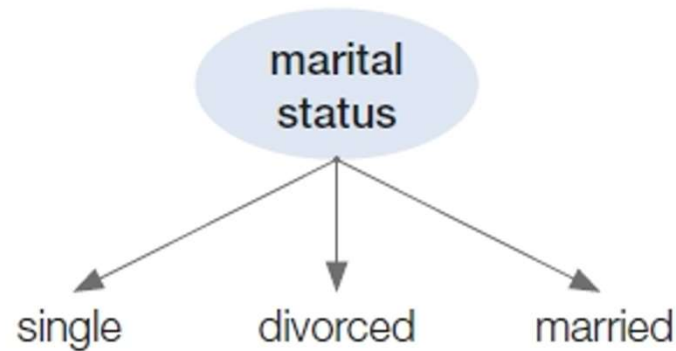
3.1 이진 분할

- CART 알고리즘의 핵심은 불확실성을 측정하는 기준 값이 엔트로피에서 지니 지수로 바뀐 것
- 구현 측면에서 가장 큰 차이점은 이진 분할을 실시한다는 것
- k 가 속성 내에 있는 범주의 개수일 때, $2^{k-1} - 1$ 개의 분할이 생성됨
- 각 속성별 지니 지수 정보

$$Gini_A(D) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2)$$

의사결정트리의 확장

결혼 상태 선택지



3가지 클래스

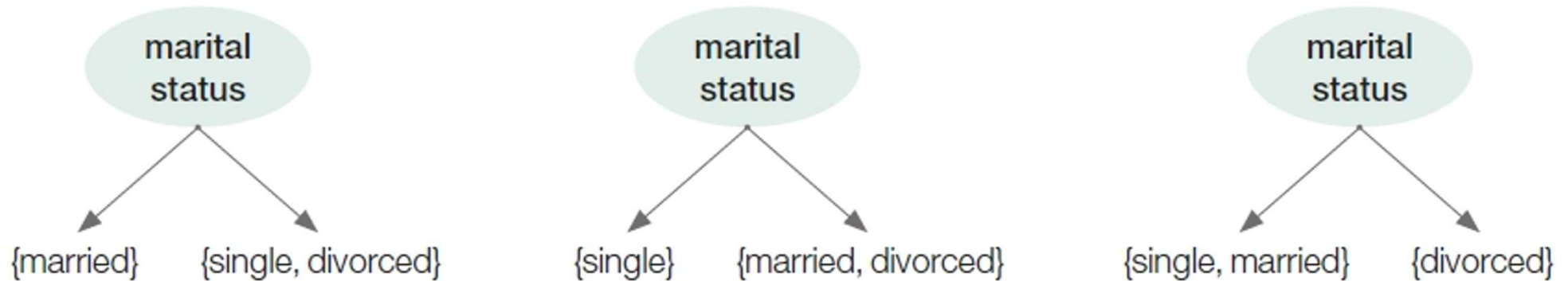


그림 10. 결혼 상태의 3가지 클래스

의사결정트리의 확장

실제 데이터에 지니 지수를 적용하여 의사결정트리 만들기

표 3. 예시 데이터

No	age	income	student	credit_rating	class_buys_computer
0	youth	high	no	fair	no
1	youth	high	no	excellent	no
2	middle_aged	high	no	fair	yes
3	senior	medium	no	fair	yes
4	senior	low	yes	fair	yes
5	senior	low	yes	excellent	no
6	middle_aged	low	yes	excellent	yes
7	youth	medium	no	fair	no
8	youth	low	yes	fair	yes
9	senior	medium	yes	fair	yes
10	youth	medium	yes	excellent	yes
11	middle_aged	medium	no	excellent	yes
12	middle_aged	high	yes	fair	yes
13	senior	medium	no	excellent	no

의사결정트리의 확장

- age, credit, income, student의 4가지 속성들을 정의

$$Gini_{age}(D) \quad Gini_{credit}(D) \quad Gini_{income}(D) \quad Gini_{student}(D)$$

- age 속성에 3가지 클래스가 존재하기 때문에 3가지 종류의 이진 분할 경우의 수로 나눌 수 있음

$$age \in \{youth, middle_aged, senior\}$$

$$age-1 \in \{youth\} = \{middle_aged, senior\}$$

$$age-2 \in \{middle_aged\} = \{youth, senior\}$$

$$age-3 \in \{senior\} = \{youth, middle_aged\}$$

의사결정트리의 확장

표 4. age가 youth인 경우 데이터

No	age	income	student	credit_rating	class_buys_computer
0	youth	high	no	fair	no
1	youth	high	no	excellent	no
7	youth	medium	no	fair	no
8	youth	low	yes	fair	yes
10	youth	medium	yes	excellent	yes

표 5. age가 youth가 아닌 경우 데이터

No	age	income	student	credit_rating	class_buys_computer
2	middle_aged	high	no	fair	yes
3	senior	medium	no	fair	yes
4	senior	low	yes	fair	yes
5	senior	low	yes	excellent	no
6	middle_aged	low	yes	excellent	yes
9	senior	medium	yes	fair	yes
11	middle_aged	medium	no	excellent	yes
12	middle_aged	high	yes	fair	yes
13	senior	medium	no	excellent	no

의사결정트리의 확장

$$Gini_{age_1}(D) = \frac{5}{14}Gini(D_1) + \frac{9}{14}Gini(D_2) = 0.393$$

$$Gini(D_1) = 1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2$$

$$Gini(D_2) = 1 - \left(\frac{7}{9}\right)^2 - \left(\frac{2}{9}\right)^2$$

- age_1, age_2, age_3 각각 연산하면 0.393, 0.357, 0.457
- age 속성의 지니 지수는 이 중 가장 작은 값인 0.357

$$Min(Gini_{age_i}) = 0.357$$

$$Min(Gini_{income_i}) = 0.443$$

$$Min(Gini_{credit}) = 0.429$$

$$Min(Gini_{student}) = 0.367$$

의사결정트리 알고리즘의 다양한 변형

트리 가지치기

- 클래스의 마지막 노드인 잎 노드(leaf node)의 개수를 개발자가 직접 결정
 - 1개로 이루어진 잎 노드가 많을 경우 과대적합되어 있는 상태
 - 잎 노드의 개수와 관계 없이 해당 가지에 불확실성이 너무 높을 경우 의사결정트리의 성능에 문제를 줄 수 있음
- **트리 가지치기(tree pruning)** : 의사결정트리의 마지막 노드의 개수를 지정하여 트리의 깊이를 조정하는 방법

의사결정트리 알고리즘의 다양한 변형

- **사전 가지치기(pre,-pruning)** : 처음 트리를 만들 때 트리의 깊이나 마지막 노드의 최소 개수 등을 사전에 결정하여 입력
 - 데이터 분석가가 하이퍼 매개변수로 모든 값을 입력해야 하는 점이 어려움
 - 계산 효율이 좋고 작은 데이터셋에서도 쉽게 작동
 - 사용자가 중요한 속성 값을 놓치거나 과소적합 문제 발생할 수 있다

의사결정트리 실습

9-1. entropy.py

```
1  import matplotlib.pyplot as plt
2  from sklearn.datasets import load_iris
3  from sklearn.tree import DecisionTreeClassifier, plot_tree
4
5  # [Part 1 & 2: Classification Tree Comparison]
6  # ID3/C4.5 (Entropy) vs CART (Gini Index)
7
8  # Load Dataset
9  iris = load_iris()
10 X = iris.data[:, 2:] # Use Petal length and width for visualization
11 y = iris.target
12
13 # 1. ID3/C4.5 Style (Using Entropy)
14 # Note: scikit-learn implements a version of CART that supports both Gini and Entropy.
15 # It performs binary splits even with entropy.
16 tree_entropy = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
17 tree_entropy.fit(X, y)
18
19 # 2. CART Style (Using Gini Index)
20 tree_gini = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
21 tree_gini.fit(X, y)
22
23 # Visualization
24 fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 8))
25
26 # Entropy Tree
27 plot_tree(tree_entropy, filled=True, feature_names=iris.feature_names[2:],
28           class_names=iris.target_names, ax=axes[0])
29 axes[0].set_title("Entropy-based Tree (ID3/C4.5 Concept)", fontsize=16)
30
31 # Gini Tree
32 plot_tree(tree_gini, filled=True, feature_names=iris.feature_names[2:],
33           class_names=iris.target_names, ax=axes[1])
34 axes[1].set_title("Gini-based Tree (Standard CART)", fontsize=16)
35
36 plt.tight_layout()
37 plt.show()
```

의사결정트리 알고리즘의 다양한 변형

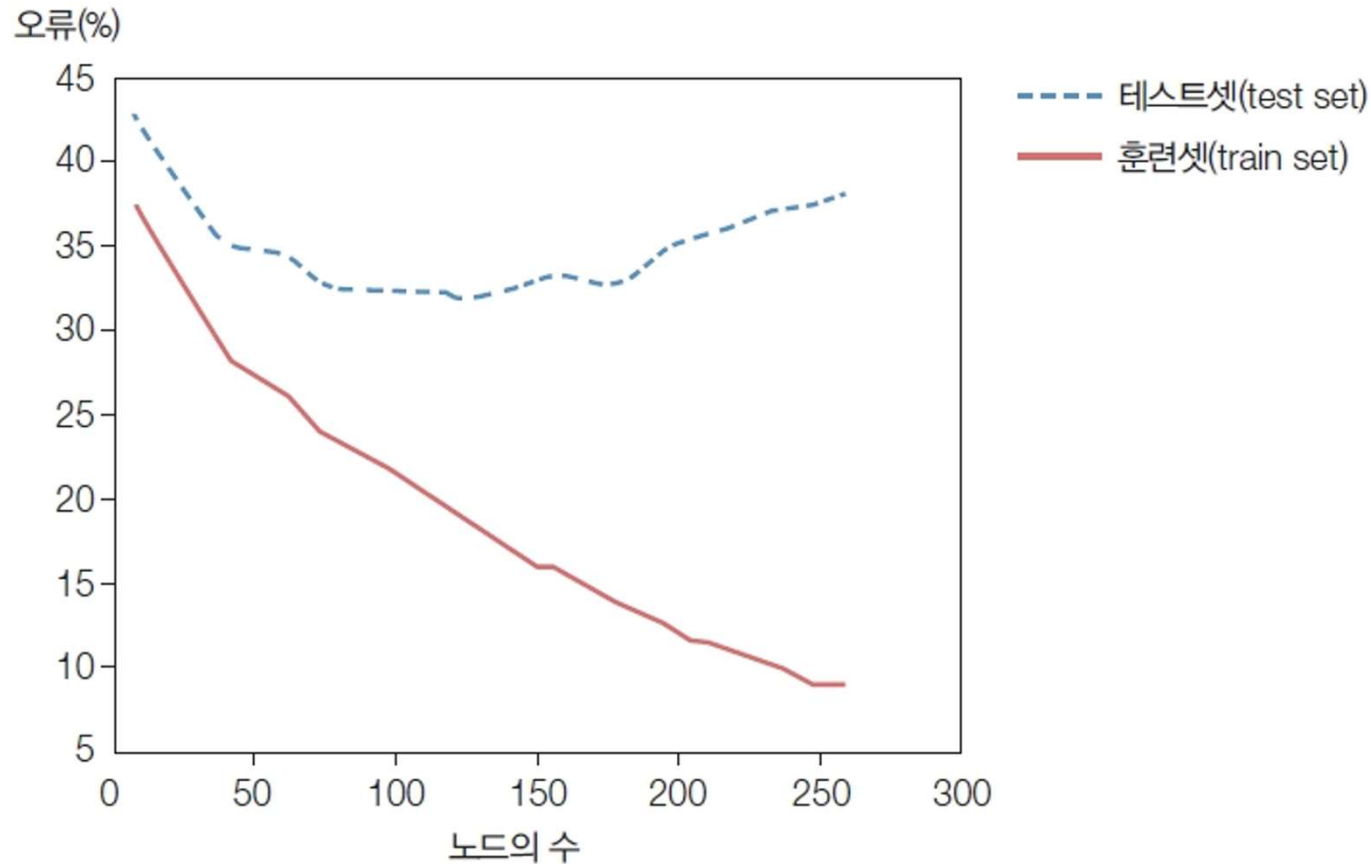


그림 11. 사후 가지치기에서 훈련셋과 테스트셋의 성능 비교

- 전체 노드 개수를 조정하면서 훈련셋과 테스트셋 성능 비교

의사결정트리 알고리즘의 다양한 변형

연속형 변수 나누기

표 6. 샘플데이터

	ID	STREAM	SLOPE	ELEVATION	VEGETATION
0	1	False	steep	3900	chapparal
1	2	True	no	300	riparian
2	3	True	steep	1500	riparian
3	4	False	steep	1200	chapparal
4	5	False	flat	4450	conifer
5	6	True	steep	5000	conifer
6	7	True	steep	3000	chapparal

의사결정트리 알고리즘의 다양한 변형

- 연속형 데이터를 나누는 기준
 - 모든 데이터를 기준으로 하여 데이터를 나누기 : 너무 많은 기준점이 생겨 과대적합 문제가 발생하거나 분류의 정확도가 떨어짐
 - 통계적 수치로 중위값이나 4분위수를 기준으로 나누기 : 25%씩 데이터를 나눠서 분류 기준을 변경. 과소적합 문제가 발생하여 분류의 성능을 떨어뜨릴 수 있음
 - 가장 많이 쓰는 방법으로, γ 클래스의 값을 기준으로 해당 값이 변할 때를 기준으로 삼아 분기

의사결정트리 알고리즘의 다양한 변형

- 데이터 중 분류 대상이 되는 기준점 찾기

	ID	STREAM	SLOPE	ELEVATION	VEGETATION	
1	2	True	moderate	300	riparian	
3	4	False	steep	1200	chapparal	(1)
2	3	True	steep	1500	riparian	(2)
6	7	True	steep	3000	chapparal	(3)
0	1	False	steep	3900	chapparal	
4	5	False	flat	4450	conifer	(4)
5	6	True	steep	5000	conifer	

그림 12. `pd_data.sort_values("ELEVATION")`

- 300, 1200, 1500, 3900에서 각각 y 데이터의 라벨이 달라졌다

의사결정트리 알고리즘의 다양한 변형

- 데이터를 자르는 기준값 정하기 : 구간별 경계 평균값

	ID	STREAM	SLOPE	ELEVATION	VEGETATION	
1	2	True	moderate	300	riparian	
3	4	False	steep	1200	chapparal	750
2	3	True	steep	1500	riparian	1350
6	7	True	steep	3000	chapparal	2250
0	1	False	steep	3900	chapparal	
4	5	False	flat	4450	conifer	4175
5	6	True	steep	5000	conifer	

그림 13. 데이터를 자르는 기준값 선정

의사결정트리 알고리즘의 다양한 변형

- 구간별 경계값을 기준으로 엔트로피를 산출
 - 4개의 기준점 각각의 정보 이득을 구했을 때 가장 큰 값이 ELEVATION의 대표 정보 이득이 되어 다른 속성값 정보 이득과 비교하여 최종적으로 분기가 일어나는 속성으로 선택됨

$$Gain(elev_{750}) = Info(D) - Info_{elev_{750}}(D)$$

$$Gain(elev_{1350}) = Info(D) - Info_{elev_{1350}}(D)$$

$$Gain(elev_{2250}) = Info(D) - Info_{elev_{2250}}(D)$$

$$Gain(elev_{4175}) = Info(D) - Info_{elev_{4175}}(D)$$

- 위 계산 결과는 각각 0.3059, 0.1813, 0.5916, 0.8631이고
STREAM, SLOPE 속성의 정보 이득은 각각 0.3059, 0.5774로 가장 먼저 ELEVATION 이 4175인 값을 기준으로 트리를 분기

의사결정트리 알고리즘의 다양한 변형

회귀 트리

- 회귀 트리(regression tree) : Y 데이터의 값이 연속형일 때의 의사결정트리 생성 방법

연속형 속성 분기의 특징

명목 속성과는 달리 여러 번 재사용이 가능하다. 경계값을 기준으로 여러 번의 분기를 할 수 있다.

처음에는 ELEVATION 4175의 값을 기준으로 분기를 했다면 동일하게 맨 마지막에는 2250의 값으로 분기를 할 수 있다.

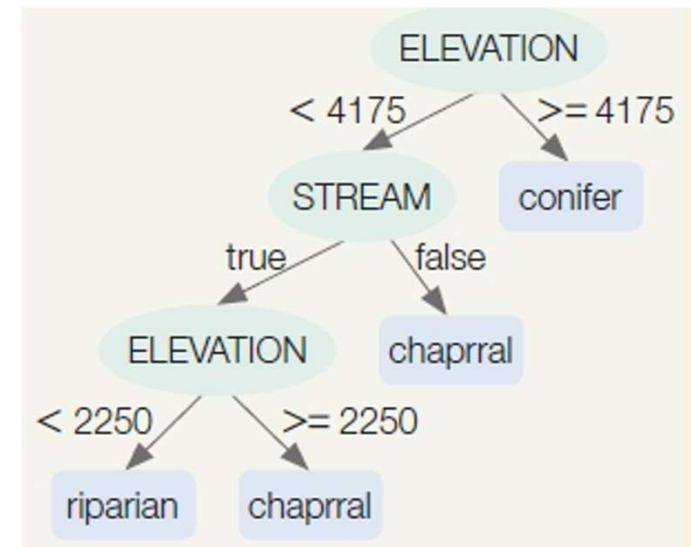


그림 14. 연속형 속성 분기의 특징

의사결정트리 알고리즘의 다양한 변형

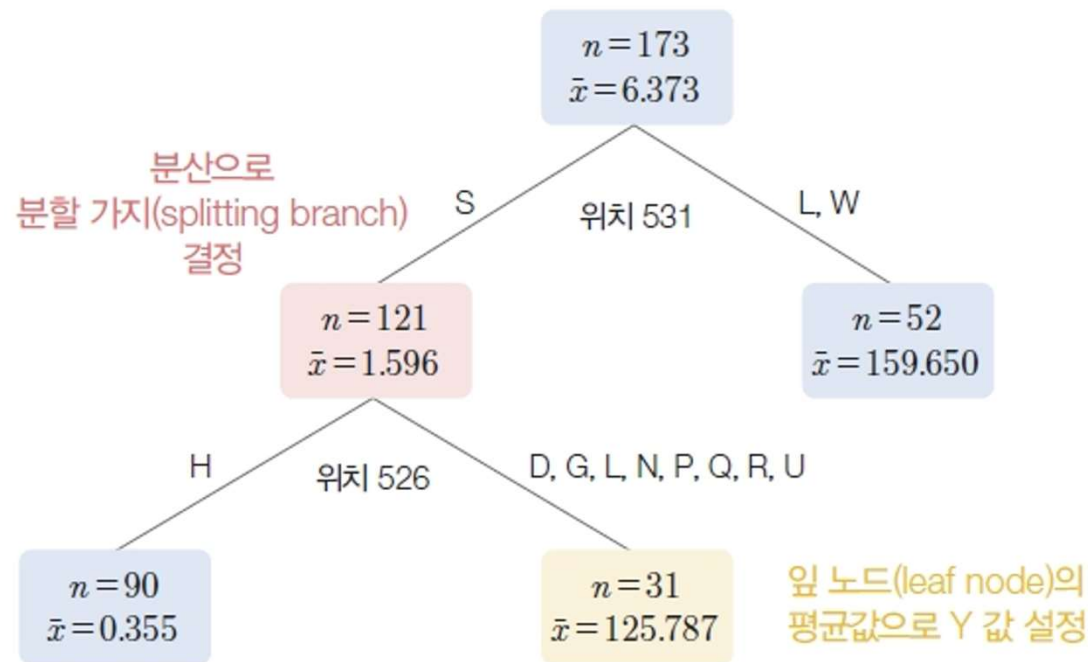


그림 15. 연속형 속성 분기의 특징

- Y 값의 각 속성별 분산이 얼마나 작은지를 측정
- 최종 결과 노드에서는 결과 노드들의 Y 평균값으로 최종 예상치를 반환

의사결정트리 알고리즘의 다양한 변형

$$\text{var}(D) = \frac{\sum_{i=1}^n (y^{(i)} - \bar{y})^2}{n-1} \quad \text{where } \bar{y} = \frac{\sum_{i=1}^n y^{(i)}}{n}$$

- 속성별 분산 구하기 : 각 클래스값들의 분산을 구하고 해당 클래스가 가진 데이터 개수의 비율만큼 곱함

$$\operatorname{argmax}_{attr \in data} \sum_{l \in \text{levels}(attr)} \frac{|D_{attr=l}|}{|D|} \times \text{var}(D_{attr=l})$$

의사결정트리 알고리즘의 다양한 변형

표 7. 연속형 값 예측을 위한 데이터

	SEASON	WORK_DAY	RENTALS		SEASON	WORK_DAY	RENTALS
0	winter	False	800	6	summer	True	3000
1	winter	False	826	7	summer	True	5800
2	winter	True	900	8	autumn	False	6200
3	spring	False	2100	9	autumn	False	2910
4	spring	True	4740	10	autumn	True	2880
5	spring	True	4900	11	autumn	False	2820

- SEASON 속성이 WORK_DAY에 비해서 분산이 낮으므로 해당 값으로 분기가 일어남

$$Var_{SEASON} = \frac{3}{12} * 618133 + \frac{3}{12} * 698033 + \frac{3}{12} * 910308 + \frac{3}{12} * 673 = 2227148$$

$$Var_{WORK_DAY} = \frac{6}{12} * 1974105 + \frac{6}{12} * 1593393 + \frac{3}{12} = 3567498$$

의사결정트리 알고리즘의 다양한 변형

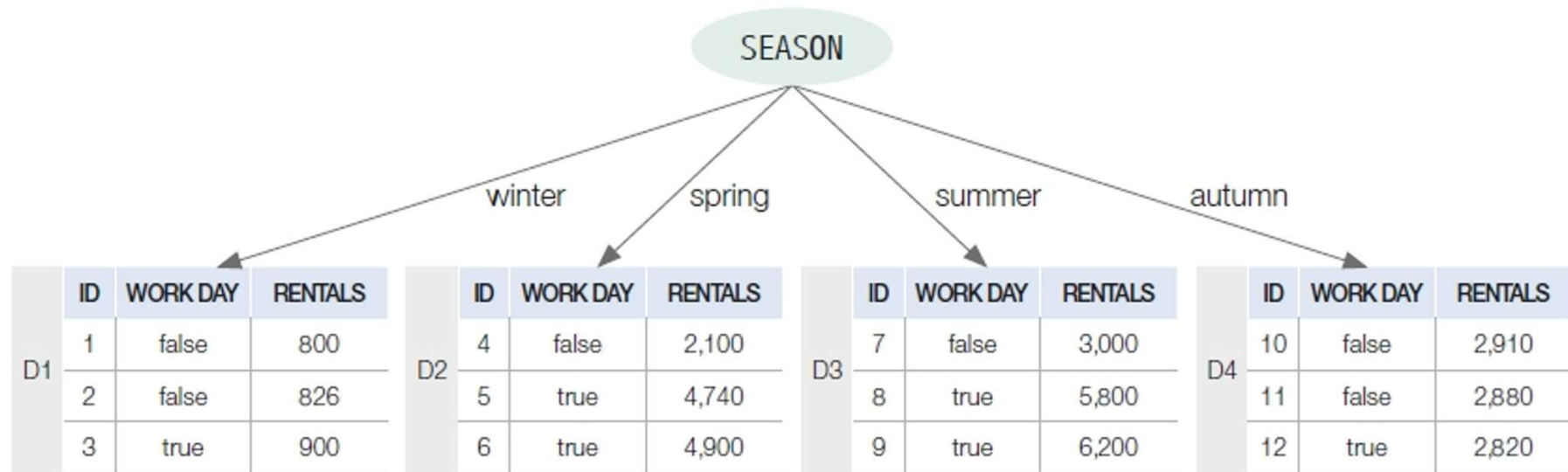


그림 16. SEASON을 기준으로 4개의 분기 발생

- 각 분기별로는 WORK_DAY로 데이터를 나누고, 나누어진 데이터는 평균값으로 최종 예측값이 반환됨
 - summer를 기준으로 분기가 일어난 후 WORK_DAY가 True인 분기에서 RENTALS 값이 5,800과 6,200이어서 평균값 6,000이 예측치로 반환됨

의사결정트리 실습

9-2. regression_tree.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.tree import DecisionTreeRegressor, plot_tree
4
5 # [Part 3: Regression Tree Example]
6 # Minimizing Variance (MSE)
7 # Generate Synthetic Data (Sine wave with noise)
8 np.random.seed(42)
9 X = np.sort(5 * np.random.rand(80, 1), axis=0)
10 y = np.sin(X).ravel()
11 y[::5] += 3 * (0.5 - np.random.rand(16))
12
13 # Train Regression Trees with different depths
14 regr_1 = DecisionTreeRegressor(max_depth=2)
15 regr_2 = DecisionTreeRegressor(max_depth=5)
16 regr_1.fit(X, y)
17 regr_2.fit(X, y)
18
19 # Predict
20 X_test = np.arange(0.0, 5.0, 0.01)[:, np.newaxis]
21 y_1 = regr_1.predict(X_test)
22 y_2 = regr_2.predict(X_test)
23
24 # Plotting the result
25 plt.figure(figsize=(10, 6))
26 plt.scatter(X, y, s=20, edgecolor="black", c="darkorange", label="data")
27 plt.plot(*args: X_test, y_1, color="cornflowerblue", label="max_depth=2", linewidth=2)
28 plt.plot(*args: X_test, y_2, color="yellowgreen", label="max_depth=5", linewidth=2)
29 plt.xlabel("data")
30 plt.ylabel("target")
31 plt.title("Decision Tree Regression (Step-function Prediction)")
32 plt.legend()
33 plt.show()
34
35 # Plot the tree structure (depth 2)
36 plt.figure(figsize=(12, 6))
37 plot_tree(regr_1, filled=True, feature_names=["X"])
38 plt.title("Regression Tree Structure (MSE Minimization)")
39 plt.show()
```